Tunisia AI Traffic Map

Traffic Simulation & Dataset Balancing Report

Mohamed Bensaad
ENICAR — Engineering Student

Dataset	OpenStreetMap — Gafsa, Tunisia
Previous	Feature Engineering Report
This Report	Traffic Simulation Engine + Dataset Balancing
Output	291,657 balanced labeled rows ready for training
Author	Mohamed Bensaad

1. Context & Where We Are

This is the fourth document in the Tunisia AI Traffic Map project series. After extracting, cleaning, and engineering features from the Gafsa road network, the next challenge was clear: we have road features but no congestion labels. A supervised machine learning model cannot train without labeled data — it needs to know the correct answer for each training example.

Since no real traffic dataset exists for Tunisia, this step builds a Traffic Simulation Engine that generates realistic congestion labels based on road structure, time of day, and Tunisia-specific traffic patterns.

Document	Content	Status
Project Introduction	Project overview, goals, tech stack	Complete
Data Analysis & Cleaning	OSMnx extraction, missing value handling	Complete
Feature Engineering	CPI, RCS, encoding, time features	Complete
Simulation & Balancing (this)	Labels generation + dataset balancing	Complete
Model Training & Validation	XGBoost training and evaluation	Next

2. Why We Need a Simulation Engine

Real traffic datasets for Tunisia simply do not exist at a national level. Uber Movement covers only Tunis city. Kaggle has nothing specific. OpenStreetMap provides road geometry but zero congestion measurements.

Key Insight: This is not a weakness of the project — it is the core engineering challenge. Building a physics-informed simulation engine that encodes domain knowledge is exactly what data scientists do when real data is unavailable. Many real-world AI products start this way.

The simulation engine uses the Congestion Pressure Index we computed in the previous step as the base score for each road segment, then adjusts it based on time context to generate a realistic congestion label.

3. How the Simulation Engine Works

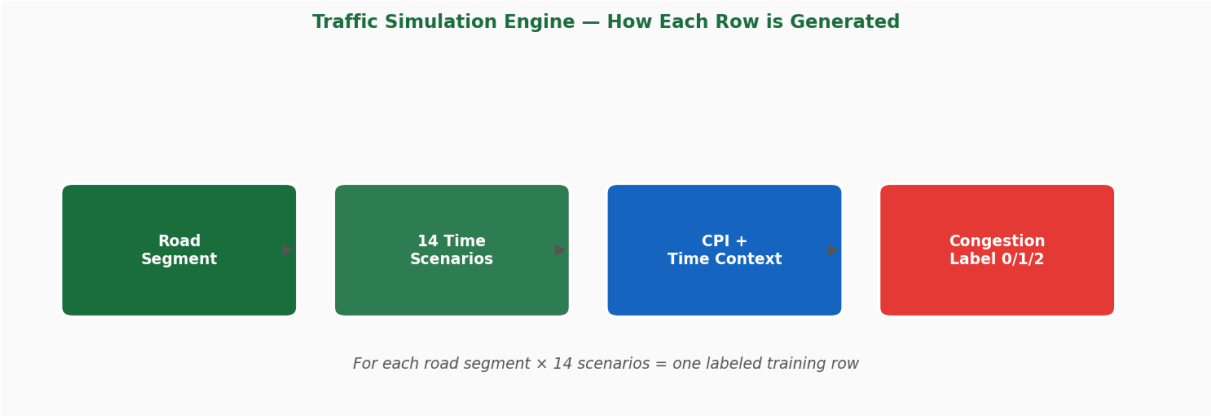

Figure 1 — For each road segment, 14 time scenarios are applied to generate 14 labeled training rows

Step 1 — Define Time Scenarios

Instead of adding time features based only on the current moment, the engine generates 14 realistic time scenarios covering all major traffic situations throughout the week. Each road segment is processed 14 times — once per scenario.

Scenario	Day	Hour	Traffic Context
Monday morning rush	Monday	08:00	Peak congestion — daily commute
Monday midday	Monday	13:00	Normal traffic
Monday evening rush	Monday	18:00	Peak congestion — return home
Monday night	Monday	23:00	Very low traffic
Tuesday morning rush	Tuesday	08:00	Peak congestion
Wednesday midday	Wednesday	13:00	Normal traffic
Thursday evening rush	Thursday	18:00	Peak congestion
Friday prayer	Friday	12:00	Tunisia-specific — mosque traffic
Friday evening	Friday	18:00	High congestion
Saturday morning	Saturday	10:00	Weekend — lower traffic
Saturday night	Saturday	20:00	Weekend evening
Sunday morning	Sunday	11:00	Weekend — low traffic
Sunday night	Sunday	21:00	Weekend evening
Ramadan night	Monday	22:00	Tunisia-specific — night activity

Step 2 — Compute the Congestion Score

For each road segment and each time scenario, a congestion score is computed starting from the road's CPI and adjusted based on time context:

```
score = CPI # base score from road structure

# Time adjustments
if is_rush_hour: score += 20 # rush hour increases congestion
if is_friday_prayer: score += 15 # mosque areas get more congested
if is_night: score -= 45 # roads are nearly empty at night
if is_weekend: score -= 20 # less traffic on weekends

# Road type fine-tuning
motorway: score -= 20 # handles high volume well
primary: score -= 10 # good capacity
residential: score += 5 # gets congested easily
living_street: score += 10 # very congested easily

# Controlled noise for realism
score += random.gauss(0, 8)
score = clamp(score, 0, 100)
```

Step 3 — Convert Score to Label

The final score is converted into one of three congestion labels using fixed thresholds:

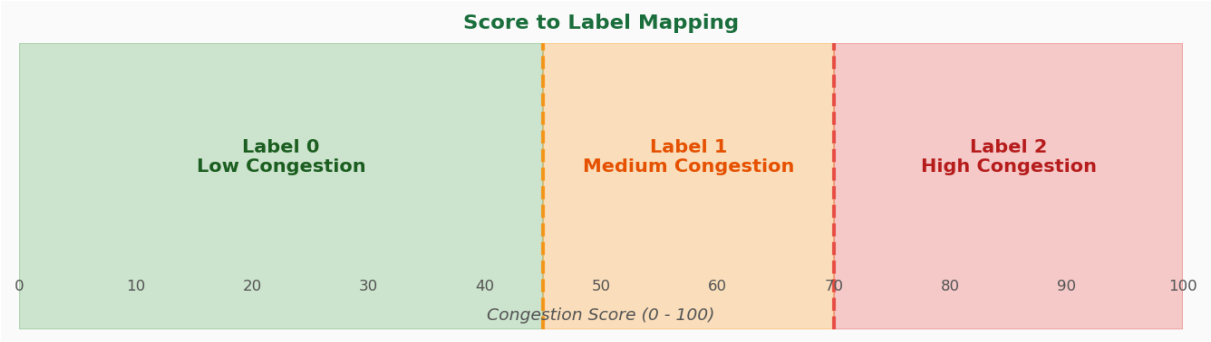

Figure 2 — Score-to-label mapping with thresholds at 45 and 70

Score Range	Label	Meaning	Map Color
0 — 44	0	Low Congestion	Green
45 — 69	1	Medium Congestion	Orange
70 — 100	2	High Congestion	Red

4. The Imbalance Problem

After running the simulation on all Gafsa road segments across 14 time scenarios, the raw label distribution showed a serious imbalance problem:

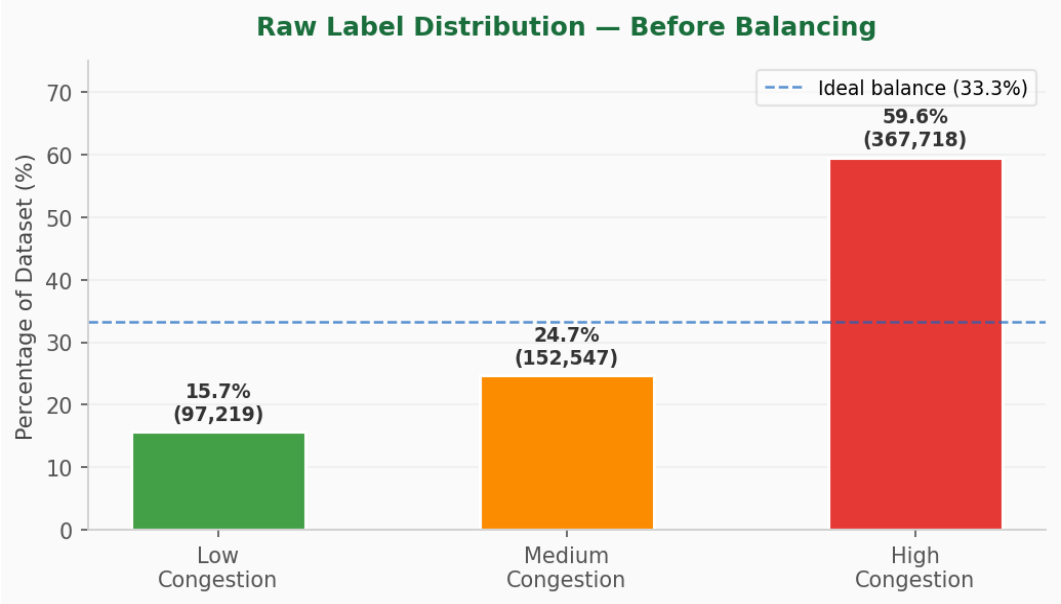

Figure 3 — Raw label distribution before balancing — High congestion dominates at 59.6%

Label	Rows	Percentage	Problem
0 — Low	97,219	15.7%	Too few — model will ignore this class
1 — Medium	152,547	24.7%	Too few — underrepresented
2 — High	367,718	59.6%	Too many — model will always predict High

Note: *Why did this happen? Gafsa has mostly residential streets with naturally high CPI values (70-90). Even with time discounts, many segments kept scoring above 70 — pushing them into the High label. This is expected for a dense city — not a bug in the engine.*

A model trained on this imbalanced data would learn a simple but wrong strategy: always predict High congestion and be correct 59.6% of the time. It would never learn to distinguish between Low, Medium, and High — making it completely useless for route comparison.

5. The Balancing Solution — Undersampling

Undersampling is the process of reducing the number of rows in the majority classes to match the size of the minority class. Instead of generating more Low and Medium rows (which would be artificial), we reduce the High rows to match the Low count.

The target size per label is the size of the smallest class — Label 0 with 97,219 rows. Both Label 1 and Label 2 are randomly sampled down to 97,219 rows each, giving a perfectly balanced dataset.

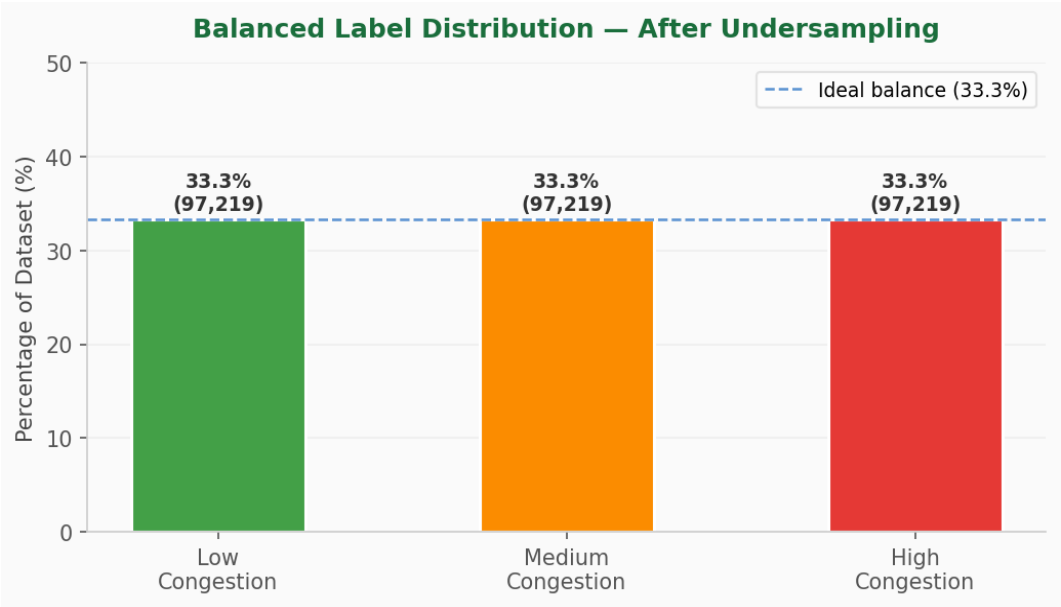

Figure 4 — Balanced label distribution after undersampling — perfect 33.3% each

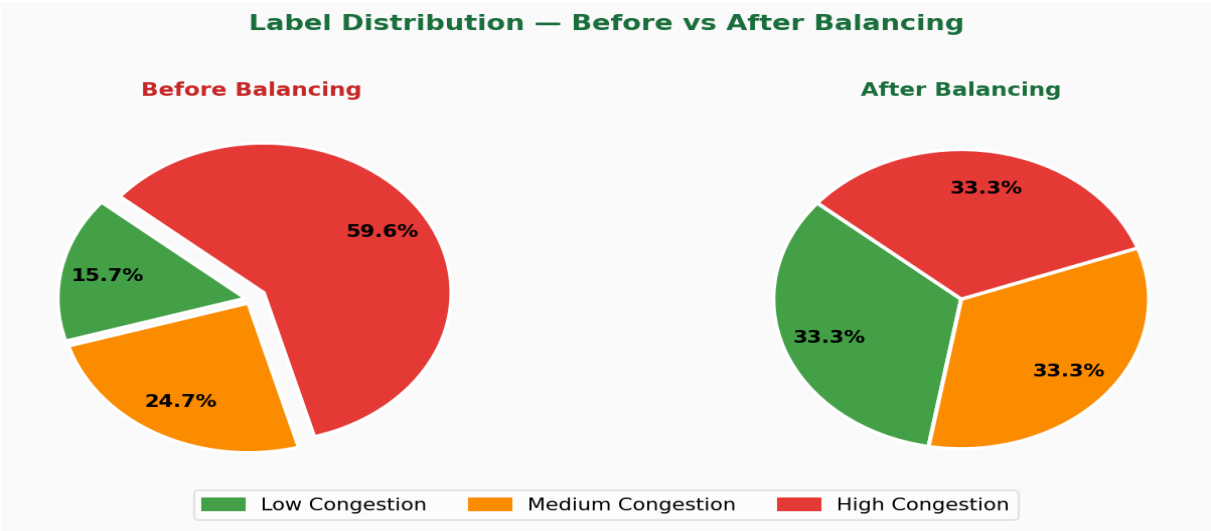

Figure 5 — Side-by-side comparison of label distribution before and after balancing

6. The Final Balanced Dataset

Property	Value
Total rows	291,657
Rows per label	97,219
Label 0 (Low)	97,219 rows — 33.3%
Label 1 (Medium)	97,219 rows — 33.3%

Label 2 (High)	97,219 rows — 33.3%
Number of features	16 input features + 1 label
Road segments source	Gafsa, Tunisia — OSMnx
Time scenarios per road	14 scenarios
Saved to	Google Drive — gafsa_balanced.csv

Key Insight: 291,657 balanced rows from a single city is already a solid training set for XGBoost. When we scale to all 24 Tunisian governorates, the dataset will grow to several million rows — making the model significantly more accurate and generalizable across all of Tunisia.

7. What Comes Next

Step	Task	Description
Next	Train XGBoost	Train classifier on df_balanced
Then	Validate Model	F1 score, confusion matrix, time-based CV
Then	Scale to 24 Governorates	Run full pipeline on all of Tunisia
Then	Export Model	Save as .pkl file
Finally	FastAPI + Frontend	Serve predictions to Leaflet.js app

Tunisia AI Traffic Map — Traffic Simulation & Balancing — Mohamed Bensaad — ENICAR